

Rising Waters

Machine Learning-Based Flood Prediction System

Project Name	Rising Waters - Machine Learning-Based Flood Prediction System
Date	04 July 2026
Author	Lahari Sri Kotipalli, B.Tech - Computer Science and Engineering
Deployment Platform	Render (Gunicorn)

Project Description

Rising Waters is a machine learning-based web application that predicts flood occurrence from historical weather and rainfall parameters. The project covers the full ML lifecycle — data preprocessing, model training, evaluation, model serialization, and deployment through a Flask web application — and now also includes user accounts, a persistent prediction history, and server-side input validation.

The system takes ten meteorological and rainfall readings — temperature, humidity, cloud cover, annual rainfall, and four seasonal rainfall breakdowns (Jan–Feb, Mar–May, Jun–Sep, Oct–Dec), average June rainfall, and subdivision rainfall — and classifies whether a flood is likely. The best-performing scikit-learn classifier and its fitted feature scaler are serialized with Joblib (floods.save and transform.save) and loaded once at Flask startup. A visitor must register or log in (with email/password or “Continue with Google”) before submitting a reading; every prediction is validated with WTForms, scored, and saved to that user's account so it can be reviewed or deleted later from a History page. Results are shown on a dedicated result page with an animated risk gauge, and the whole interface supports a persisted light/dark theme along with an ambient rain-and-lightning canvas animation.

Scenarios

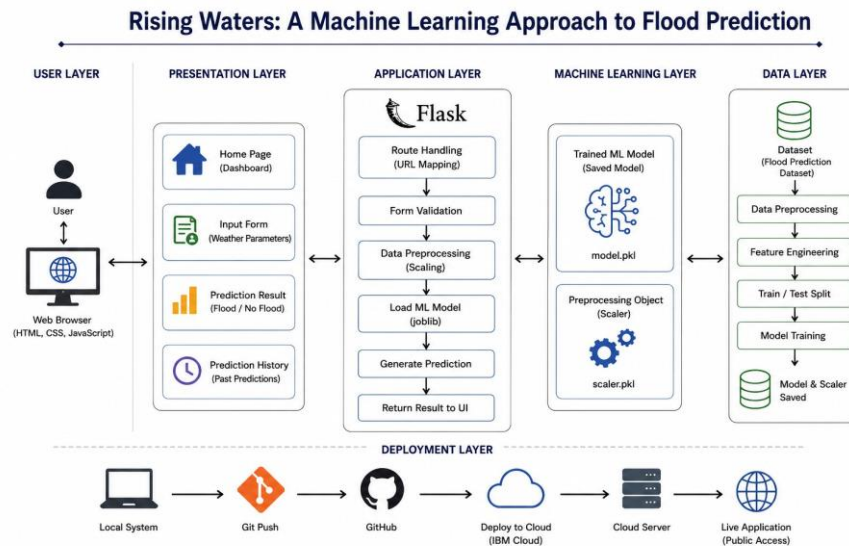
Scenario 1: A resident registers an account and checks the app during a heavy monsoon week, entering high humidity, high cloud cover, and high seasonal rainfall figures. The system returns a Flood Predicted result, with the risk gauge animating into the high range, and the reading is saved to their history.

Scenario 2: The same resident logs back in during a dry spell with low rainfall and humidity values. The system returns a No Flood Predicted result, with the gauge settling in the low range, and both readings are now visible side by side on their /history page.

Scenario 3: An evaluator tests the deployed app's /predict endpoint directly with a partial payload (missing fields) using Postman or curl while authenticated, confirming the app fails safely with a WTForms validation error instead of crashing.

Scenario 4: An anonymous visitor tries to open /predict or /history without logging in. Flask-Login redirects them to /login with a “Please log in to continue” flash message, and returns them to the page they originally requested once they authenticate.

Technical Architecture



Rising Waters uses a four-layer architecture. The **Presentation Layer** (Jinja2 templates, shared partials/nav.html and partials/flash.html, custom CSS3 with light/dark theme variables, and vanilla JavaScript) renders the input form, the risk gauge, and the rain/lightning canvas effect. The **Application Layer** (Flask routes in app.py) handles authentication, form validation via WTForms, assembles the ten readings into a DataFrame with the exact training column order, and dispatches them to the model. The **Data / Persistence Layer** (extensions.py, models.py) is a SQLAlchemy database with User and Prediction tables — SQLite locally, or Postgres in production via the DATABASE_URL environment variable — alongside the Joblib-serialized scikit-learn classifier and its fitted scaler, both loaded once at process startup rather than per-request. A thin **Security layer** (Flask-Login sessions, Flask-WTF CSRF protection, Werkzeug password hashing, and Authlib for Google OAuth) sits across the application and data layers. In production the app runs behind Gunicorn and is deployed to Render.

(Note: this diagram was produced for the original model-serving design and has not yet been redrawn to depict the authentication and persistence layers described above — treat it as an illustrative reference for the ML serving path, not a complete system diagram.)

Pre-requisites

- Python Programming Proficiency: [Python Documentation](#)
- Flask Framework Knowledge: [Flask Documentation](#)
- Flask Extensions: [Flask-SQLAlchemy](#), [Flask-Login](#), [Flask-WTF Documentation](#)
- Machine Learning Libraries: [scikit-learn](#), [Pandas](#), and [NumPy documentation](#)
- HTML, CSS, and JavaScript Skills: [W3Schools HTML/CSS/JavaScript Tutorials](#)
- Version Control with Git: [Git Documentation](#)
- Model Serialization: [Joblib Documentation](#)
- OAuth Basics: [Authlib Documentation](#) (for “Continue with Google”)
- Environment Variable Management: [python-dotenv Documentation](#)
- Deployment Basics: [Gunicorn and Render Documentation](#)

Project Workflow

The project was executed across six milestones, moving from data preparation through model training, backend and frontend development (including a later authentication pass), to public deployment and final testing.

Milestone 1: Data Preparation & Model Selection

- Activity 1.1: Collect and clean the historical rainfall and weather dataset (floods dataset.xlsx).
- Activity 1.2: Perform Exploratory Data Analysis using Pandas, Matplotlib, and Seaborn to examine feature distributions and correlations.
- Activity 1.3: Handle missing values and detect outliers in the rainfall and weather columns.
- Activity 1.4: Fit a feature scaler and define the train/test split used for model comparison.

Milestone 2: Model Training & Evaluation

- Activity 2.1: Train multiple classification models (Decision Tree, Random Forest, Gradient Boosting, KNN) on the scaled features.
- Activity 2.2: Evaluate each model using Accuracy Score, Confusion Matrix, Precision, Recall, F1-Score, and a full Classification Report.
- Activity 2.3: Select the best-performing model based on the evaluation metrics.
- Activity 2.4: Serialize the selected classifier as floods.save and the fitted scaler as transform.save using Joblib, so the Flask app can load them without depending on the training notebook.

Milestone 3: Flask Backend Development (app.py)

Activity 3.1: Define the Application's Routes

Route	Method	Login Required	Renderers / Action
/	GET	No	home.html — landing page
/register	GET, POST	No	register.html — create an account
/login	GET, POST	No	login.html — email/password sign-in
/login/google	GET	No	Redirects to Google's OAuth consent screen
/login/google/callback	GET	No	Completes Google sign-in, creates/links the account
/logout	GET	Yes	Ends the session, redirects to home
/predict	GET, POST	Yes	predict.html, or chance.html / no_chance.html on submit
/history	GET	Yes	history.html — the user's saved predictions
/history/<id>/delete	POST	Yes	Deletes one saved prediction, redirects to /history

Activity 3.2: Load the Model and Scaler at Startup (with a safe fallback)

```
MODEL_READY = True
try:
    model = joblib.load("floods.save")
    scaler = joblib.load("transform.save")
except Exception:
```

```
logger.exception("Failed to load model/scaler - prediction will be unavailable.")
model = None
scaler = None
MODEL_READY = False
```

If the model files fail to load, the app still starts — login, registration and history keep working — but /predict shows a friendly “unavailable” message instead of crashing.

Activity 3.3: Registration and Login

```
@app.route("/register", methods=["GET", "POST"])
def register():
    form = RegistrationForm()
    if form.validate_on_submit():
        user = User(username=form.username.data.strip(),
                    email=form.email.data.strip().lower())
        user.set_password(form.password.data)
        db.session.add(user)
        db.session.commit()
        login_user(user)
        flash("Account created - welcome!", "success")
        return redirect(url_for("home"))
    return render_template("register.html", form=form)
```

Login mirrors this with LoginForm, checking the hashed password via `user.check_password()`. “Continue with Google” is handled by `login_google()` / `google_callback()`, which look up or create a User by Google account ID and link it to an existing email-based account automatically if one already exists.

Activity 3.4: Prediction Route (validated, saved to history)

```
@app.route("/predict", methods=["GET", "POST"])
@login_required
def predict():
    form = PredictForm()
    if form.validate_on_submit():
        data = pd.DataFrame([form.Temp.data, form.Humidity.data,
                             form.CloudCover.data, form.Annual.data, form.JanFeb.data,
                             form.MarMay.data, form.JunSep.data, form.OctDec.data,
                             form.AvgJune.data, form.Sub.data], columns=FEATURE_COLUMNS)

        scaled = scaler.transform(data)
        prediction = model.predict(scaled)
        flood_predicted = bool(prediction[0] == 1)

        record = Prediction(user_id=current_user.id, temp=form.Temp.data,
                            humidity=form.Humidity.data, flood_predicted=flood_predicted, ...)
        db.session.add(record)
        db.session.commit()

        return render_template("chance.html" if flood_predicted else "no_chance.html")
    return render_template("predict.html", form=form, model_ready=True)
```

Activity 3.5: Input Validation & Error Handling

- All 10 fields are validated server-side by PredictForm (WTForms DataRequired + NumberRange per field) instead of being read directly from request.form.
- CSRF tokens (Flask-WTF) protect every POST form, including prediction, login, registration and the history delete action.

Milestone 4: Frontend Development

Activity 4.1: Home Page

Introduces the project with a hero section and links to the prediction form (home.html).

Activity 4.2: Authentication Pages

Added login.html and register.html, each rendering its WTForms form with inline field errors, a “Continue with Google” button (shown only when Google OAuth credentials are configured), and a shared partials/nav.html that swaps Login/Sign up links for the username, History link and Log out link once a user is authenticated.

Activity 4.3: Prediction Page

- predict.html collects all 10 model inputs, grouped into WeatherConditions, Rainfall Totals, and Sub-division Rainfall sections.
- A light/dark theme toggle is available at all times, with the choice persisted across visits via localStorage.

Activity 4.4: Result Pages

chance.html - shown when a flood is predicted; displays an animated high-risk gauge. no_chance.html - shown when no flood is predicted; displays an animated low-risk gauge. Both pages link back to the prediction form and to the new History page.

Activity 4.5: Prediction History Page

Added history.html, listing every prediction the logged-in user has made (date, result badge, temperature, humidity, annual rainfall) with a Delete action per row, backed by the /history/<id>/delete route.

Activity 4.6: Flash Messages & Ambient Canvas Animation

- partials/flash.html renders success/error/info banners after actions such as login, registration, and failed predictions.
- A rain-and-lightning canvas animation in static/main.js runs for atmosphere across all pages.

Activity 4.7: Styling

static/main.css implements the responsive layout and theme variables, now extended with styles for form field errors, flash messages, the history table, and status badges - reusing the existing panel/field/btn classes so the new pages match the original visual design.

Milestone 5: Deployment (Render)

Activity 5.1: requirements.txt

```
Flask
Flask-SQLAlchemy
Flask-Login
Flask-WTF
WTForms
email-validator
python-dotenv
Authlib
gunicorn
numpy==1.26.4
pandas==2.2.2
matplotlib
seaborn
scikit-learn==1.5.0
joblib==1.4.2
requests
```

Activity 5.2: Procfile (unchanged)

```
web: gunicorn app:app --bind 0.0.0.0:$PORT
```

Activity 5.3: Configure Environment Variables on Render

Unlike the original submission, the app now reads secrets from the environment (via python-dotenv locally, and the host's environment settings in production). On Render's dashboard, set:

Variable	Purpose
SECRET_KEY	Signs sessions and CSRF tokens — generate with <code>secrets.token_hex(32)</code>
DATABASE_URL	Postgres connection string in production (SQLite is used if unset, for local dev only)
SESSION_COOKIE_SECURE	Set to true once served over HTTPS
GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET	Optional — enables the “Continue with Google” button

Activity 5.4: Push the Repository to GitHub

```
git init
git add .
git commit -m "Add authentication, history, and validation"
git remote add origin https://github.com/Laharisrikotipalli/RisingWaters.git
git push -u origin main
```

Activity 5.5: Create / Update the Render Web Service

- Sign in to Render and choose New → Web Service (or open the existing service).
- Connect the GitHub repository containing the Rising Waters project.
- Set the Build Command to `pip install -r requirements.txt`.
- Set the Start Command to `gunicorn app:app --bind 0.0.0.0:$PORT` (or leave it to the Procfile).
- Add the environment variables from the table above under the service's Environment tab.
- Attach a managed Postgres database and set `DATABASE_URL` to it for persistent history across redeloys.

Activity 5.6: Verify the Live Deployment

Once the build finishes, Render exposes the app at its assigned URL. Open it, register a test account, and confirm both prediction outcomes (Flood Predicted and No Flood Predicted) save correctly to /history on the live instance.

Important On Render's free tier, an inactive service spins down and takes a few seconds to wake up on the next request; this is expected behavior, not an error.

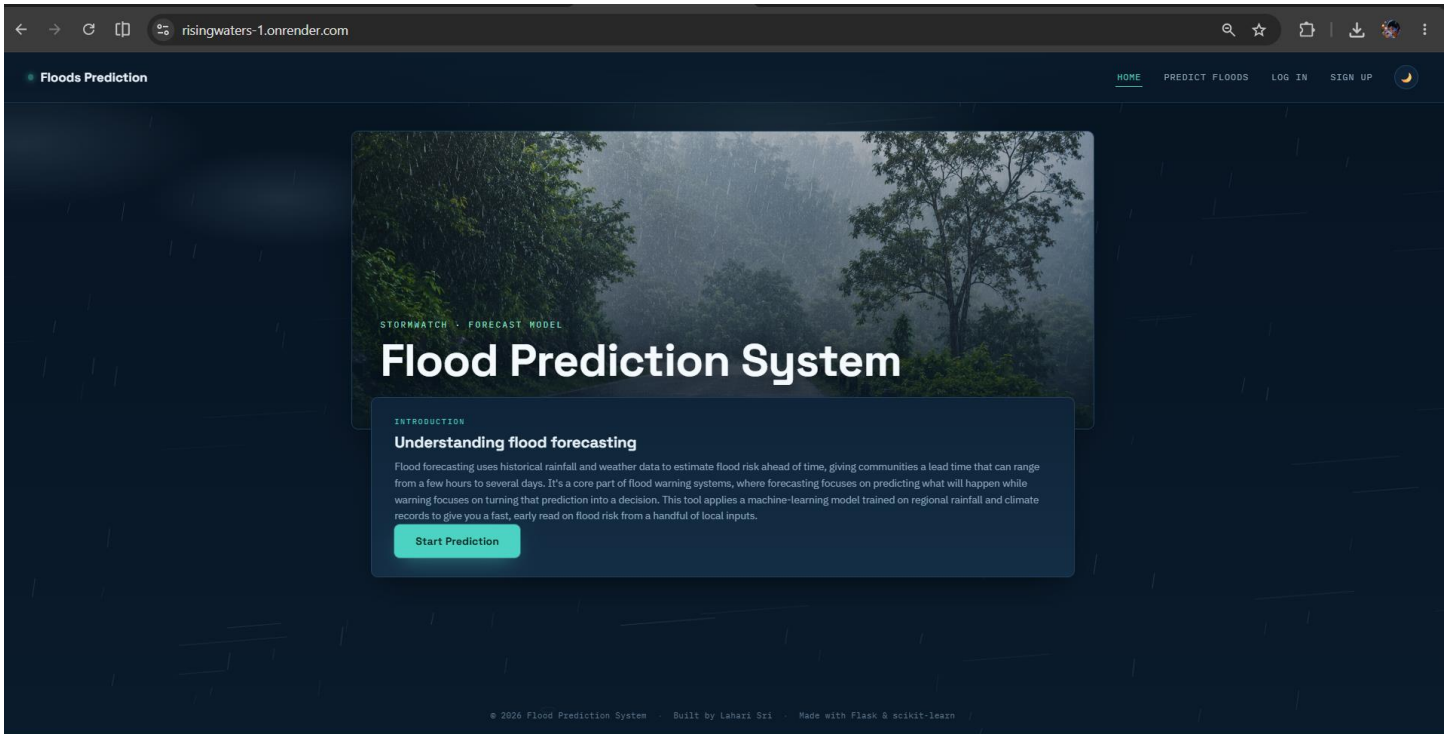
Milestone 6: Testing & Documentation

- Activity 6.1: Manually verify both prediction outcomes (flood and no-flood) using representative input values, while logged in.
- Activity 6.2: Test the /predict endpoint directly via Postman or curl, including edge cases such as missing or malformed fields, and confirm the CSRF-protected routes reject requests without a valid token.
- Activity 6.3: Exercise the authentication flow end to end - register, log out, log back in, and (where configured) sign in with Google — and confirm accounts with the same email are linked rather than duplicated.
- Activity 6.4: Verify the History page lists new predictions immediately and that deleting an entry removes only that user's own record.
- Activity 6.5: Document the setup, architecture, and workflow, and record a short demo walkthrough of the deployed app.

Exploring the Website's Web Pages

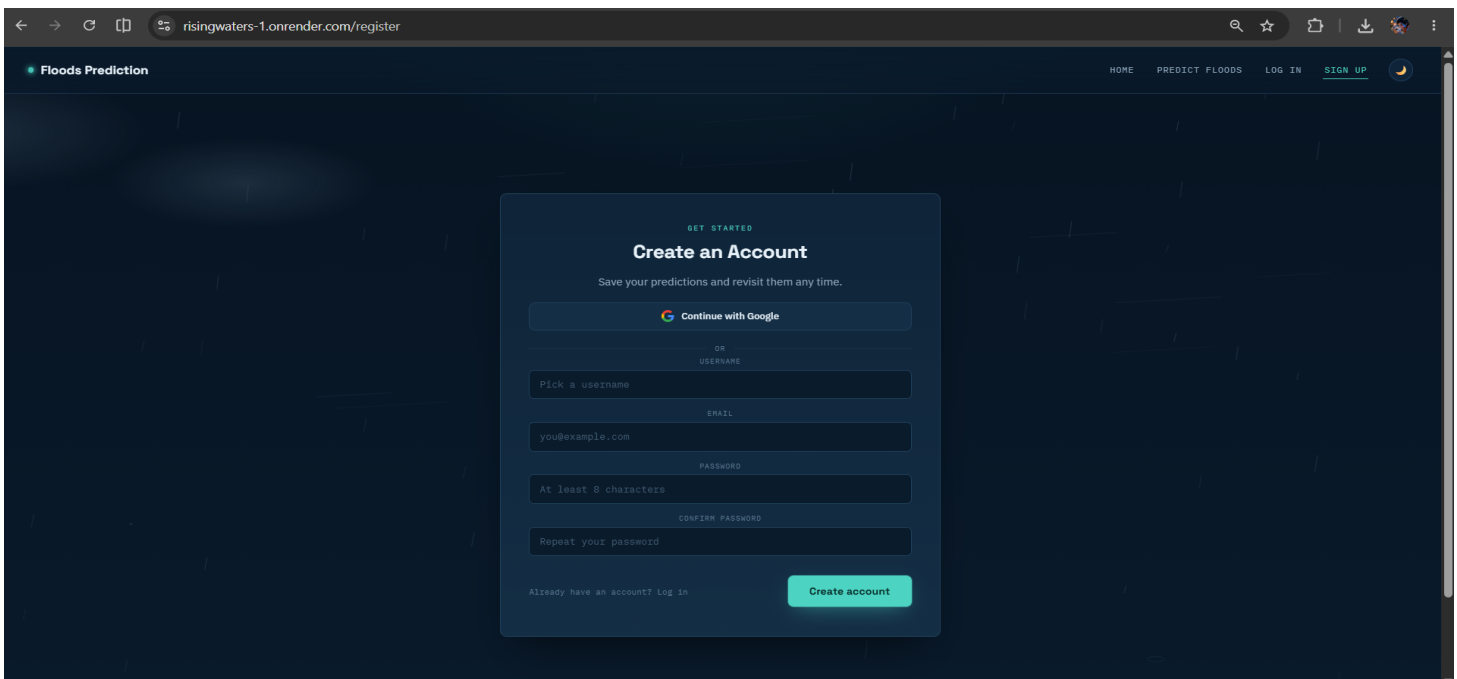
Home Page

Introduces the project with a hero section and background on flood forecasting, and a call to action linking to the prediction form (or to sign in first, for a first-time visitor).



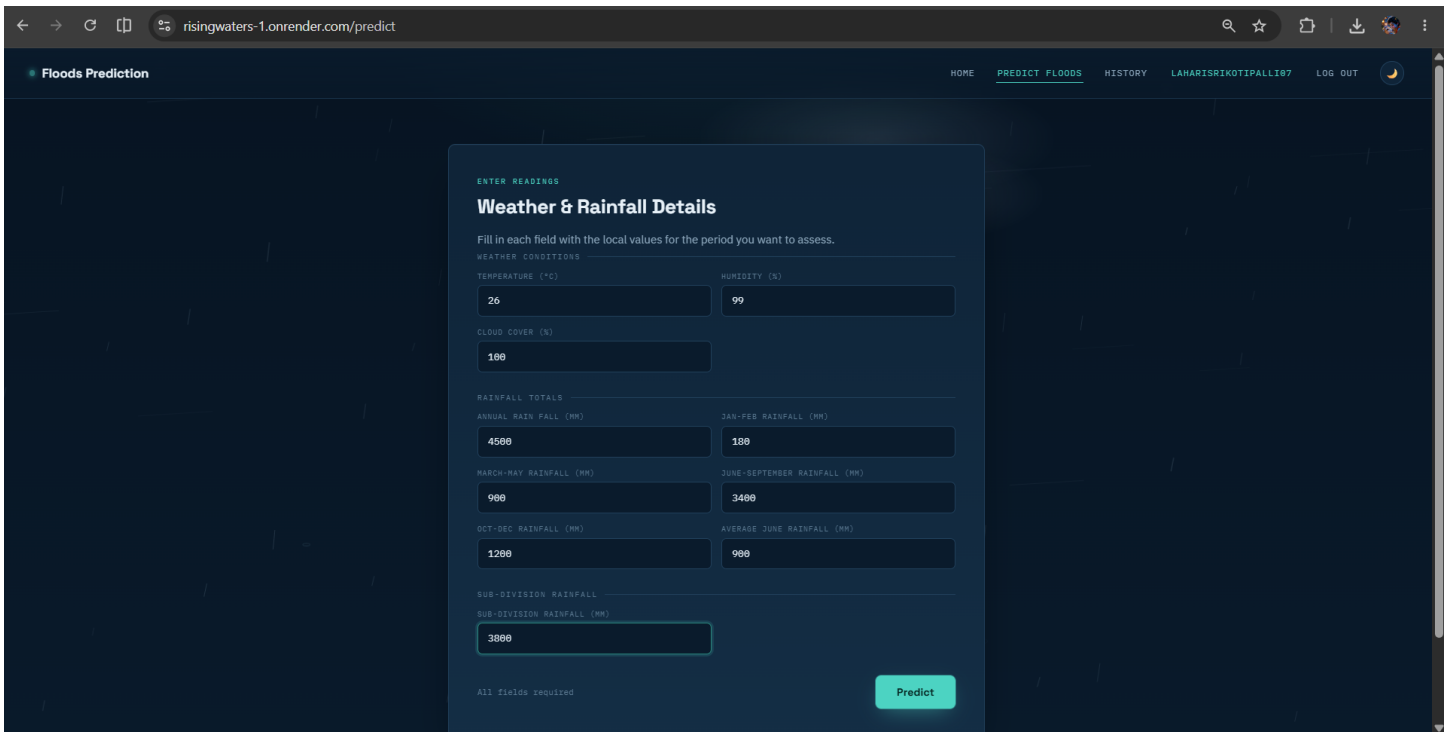
Register / Login Pages

register.html collects username, email, and password with inline validation; login.html accepts email/password or “Continue with Google”. Both share the site's nav, flash messages, and theme toggle.



Prediction Page

A form collecting all 10 model inputs, grouped into Weather Conditions, Rainfall Totals, and Sub-division Rainfall sections, with a light/dark theme toggle available at all times. Only reachable once logged in.



ENTER READINGS

Weather & Rainfall Details

Fill in each field with the local values for the period you want to assess.

WEATHER CONDITIONS

TEMPERATURE (°C) HUMIDITY (%)

CLOUD COVER (%)

RAINFALL TOTALS

ANNUAL RAIN FALL (MM) JAN-FEB RAINFALL (MM)

MARCH-MAY RAINFALL (MM) JUNE-SEPTEMBER RAINFALL (MM)

OCT-DEC RAINFALL (MM) AVERAGE JUNE RAINFALL (MM)

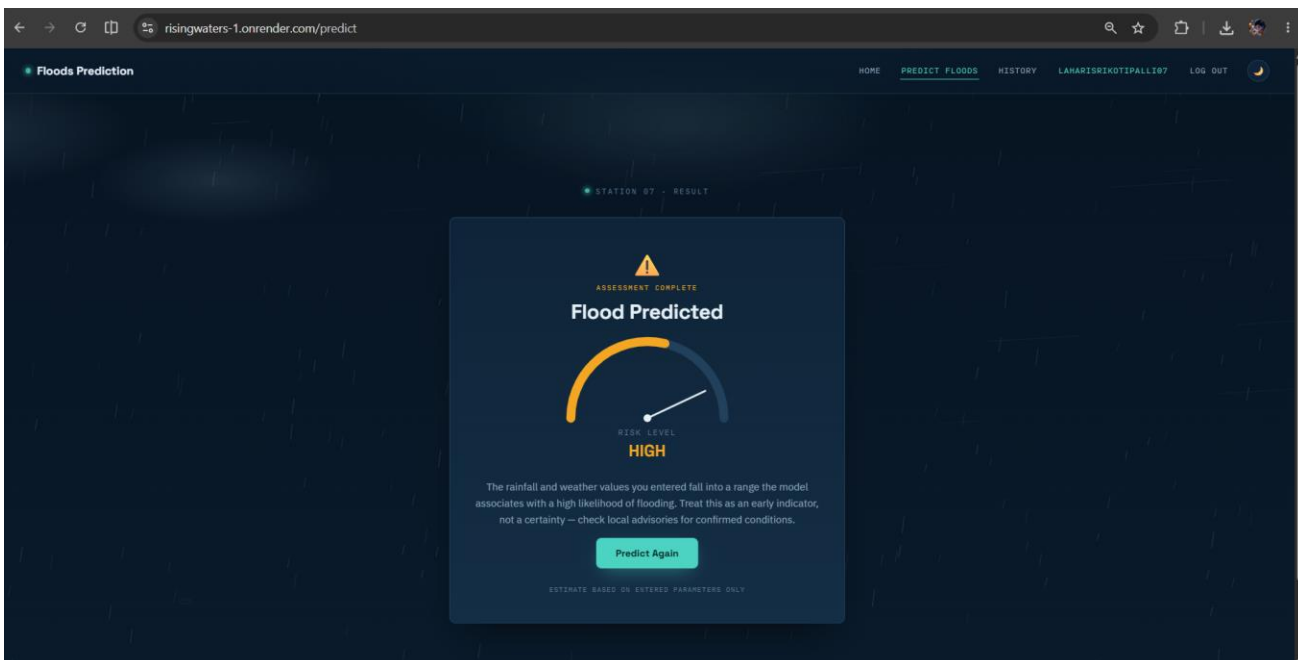
SUB-DIVISION RAINFALL

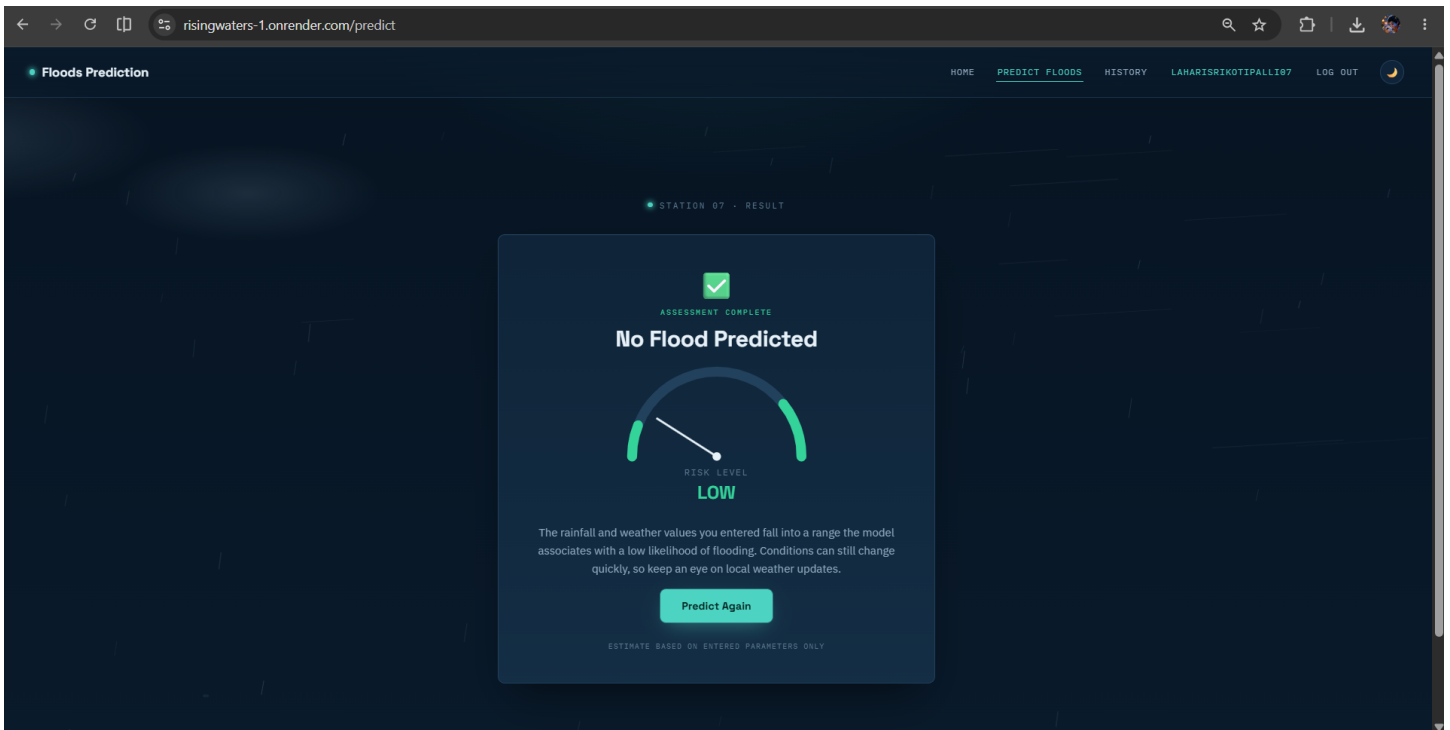
SUB-DIVISION RAINFALL (MM)

All fields required

Result Pages

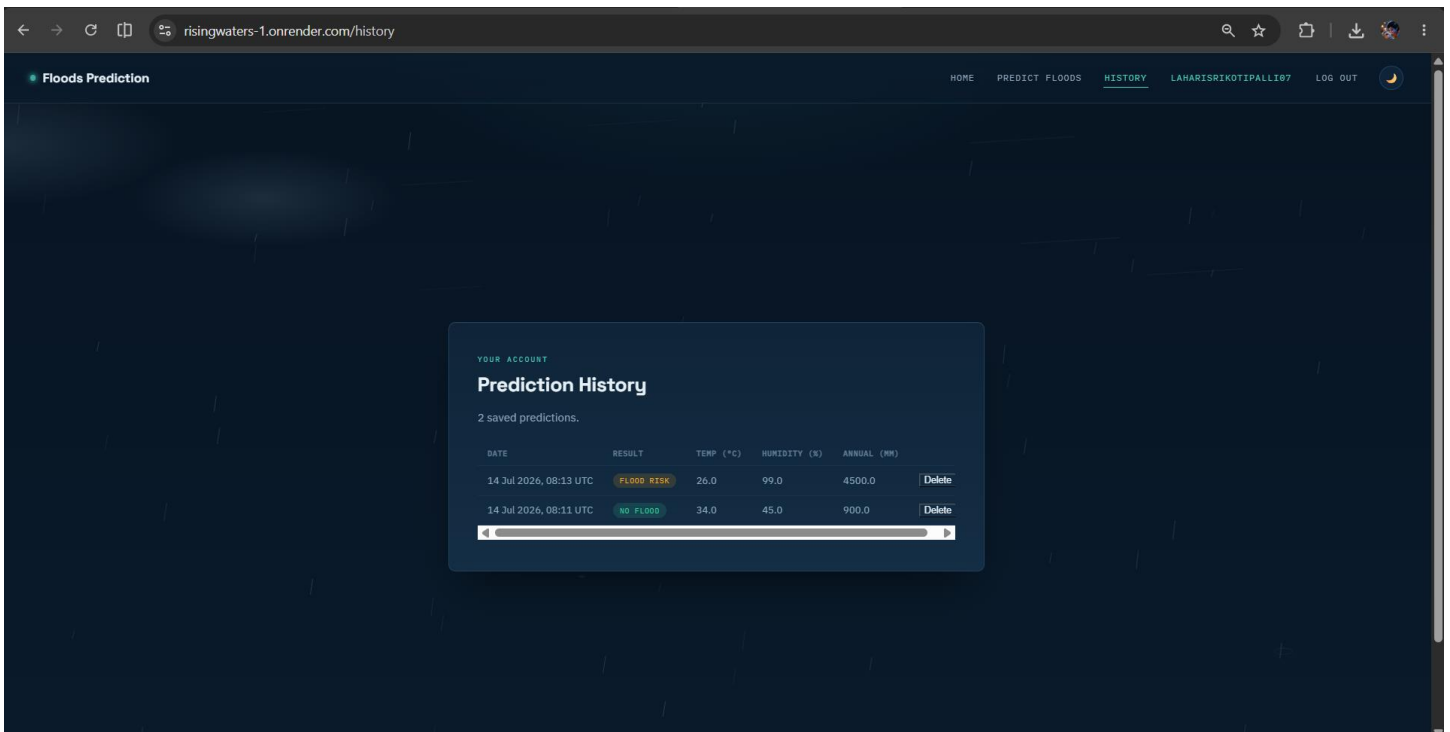
chance.html shows a Flood Predicted message with a high-risk animated gauge; no_chance.html shows a No Flood Predicted message with a low-risk gauge. Both pages link back to the prediction form and to History.





History Page

history.html lists every saved prediction for the logged-in user in a table, with a result badge and a Delete action per row.



Conclusion

Rising Waters is a machine learning platform built with Flask that predicts flood occurrence from ten weather and rainfall

readings. It compares classical classification models, serializes the best-performing pipeline with Joblib, and serves predictions instantly through a responsive, themeable web interface complete with an animated risk gauge and ambient rain effects. Since the original submission, the project has grown to include user accounts (email/password and Google OAuth), a SQLAlchemy-backed prediction history, server-side form validation, CSRF protection, and custom error pages — moving it from a stateless demo toward a small multi-user application. The project, which spanned data preparation, model training, Flask development, an authentication layer, and deployment to Render, demonstrates how a compact ML pipeline can be packaged into a usable, persistent public tool. Looking ahead, Rising Waters has clear room to grow — real-time weather API integration, river and dam water-level monitoring, satellite data, deep learning-based forecasting, password-reset and rate-limiting on auth, a JSON API endpoint, and a mobile companion app are all natural next steps toward a more comprehensive flood early-warning system.